

Agenda

→ Factory Design Pattern

- factory method ✓✓
- Abstract factory
- practical factory

User management Application

UserService → creates and updates users in the database.

```
class UserService {  
    Database db;  
    createVuser() {  
        Query q = db.createQuery("INSERT INTO users...");  
        q.execute();  
    }  
    updateVuser() {  
        Query q = db.createQuery("UPDATE users SET...");  
        q.execute();  
    }  
}
```

- ① Create a query using database
- ② Executes the query.

→ Database can be an interface or abstract class. (if it has some shared state)

Why?

- ① Dependency Inversion Principle.
- ② Flexibility.

User Service

depends on

<< Database >> (Interface)

MySQL Database

Mongo Database (concrete implementations)

<< Query >>

MySQLQuery

MongoQuery

MySQL Database. createQuery();
↓
Factory method.

MySQL Database

connect()

changePoolSize()

createQuery() → MySQLQuery

Mongo Database

connect()

changePoolSize()

createQuery() → MongoQuery

Adidas factory

↓
shoes.

Bread for
b
Bread.

interface Database {

void connect();
void changePoolSize();
String getVersion();

} Database operations

Query createQuery(String s);
Transaction createTransaction();
--
--

} → object creation

Abstract factory

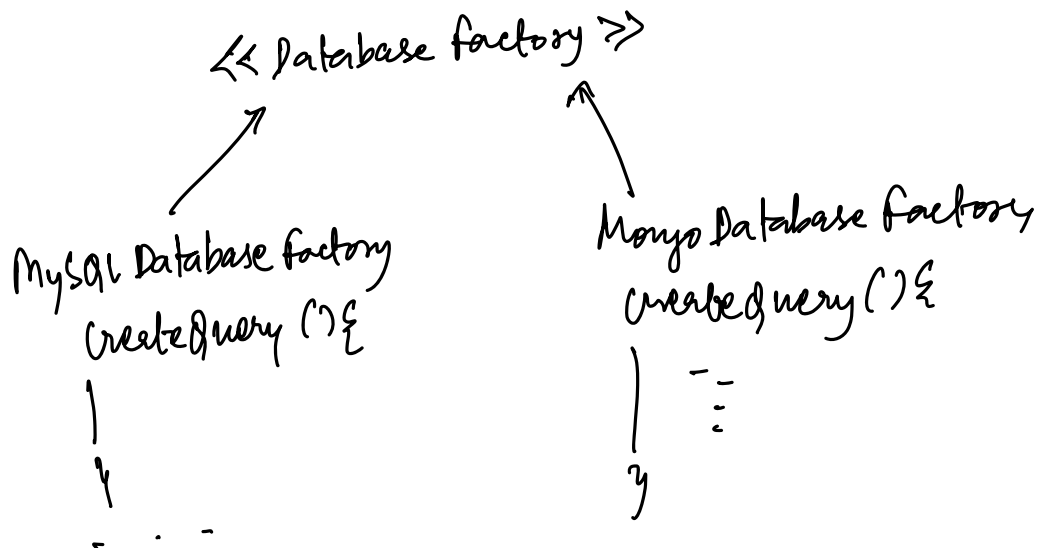
BEFORE (SRP violation):

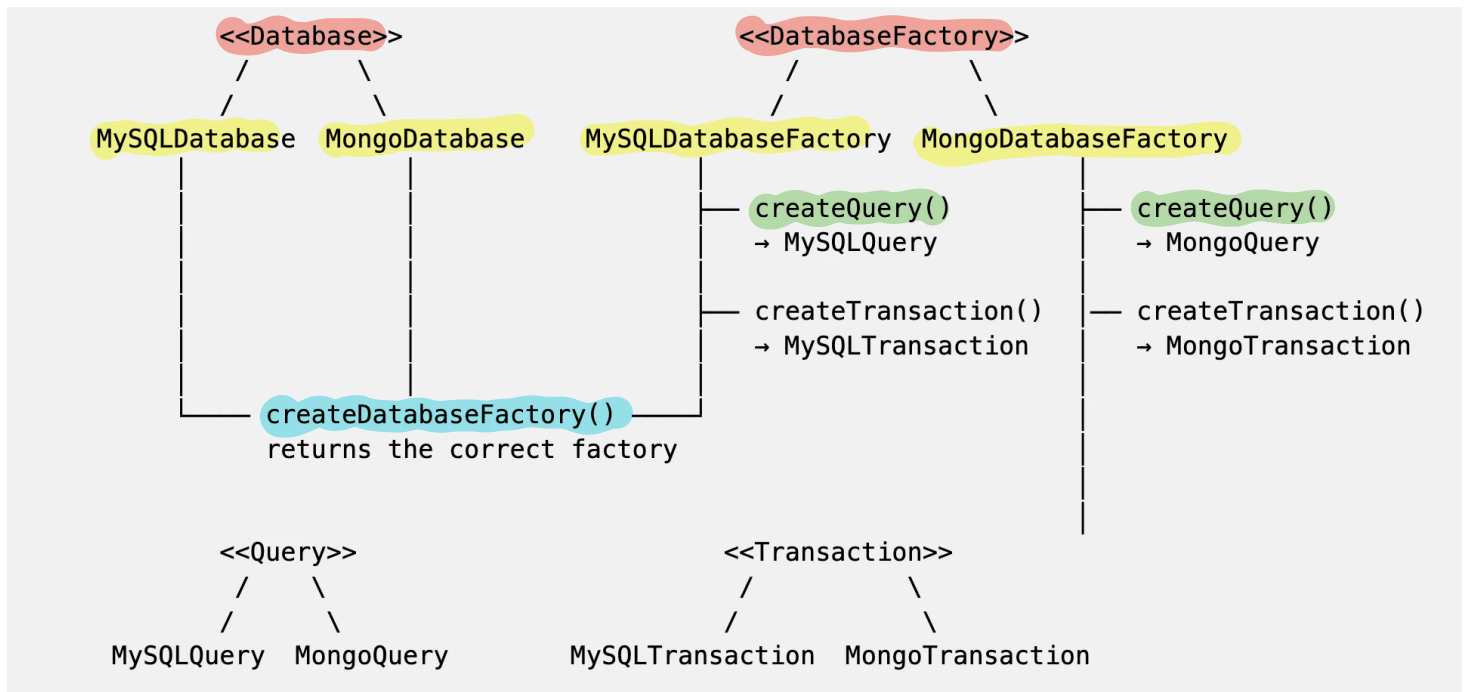
```
<<Database>>
| connect()
| changePoolSize()
| getVersion()
| createQuery() ← move out
| createTransaction() ← move out
```

AFTER (SRP clean):

```
<<Database>>
| connect()
| changePoolSize()
| getVersion()
| createDatabaseFactory() ← bridge method
```

```
<<DatabaseFactory>>
| createQuery()
| createTransaction()
```





→ Practical Use case - UI libraries.

```
class Flutter {  
  Button createButton(String platform) {  
    if (platform.equals("Android")) {  
      return new AndroidButton();  
    } else if (platform.equals("iOS")) {  
      return new IosButton();  
    }  
    return null;  
  }  
  
  Menu createMenu(String platform) {  
    if (platform.equals("Android")) {  
      return new AndroidMenu();  
    } else if (platform.equals("iOS")) {  
      return new IosMenu();  
    }  
    return null;  
  }  
  
  // ... more UI components, each with the same if/else!  
}
```

DRY
→ Don't repeat
yourself.

Desktop,
Web.

Violates ocp
violating SRP.

You have to refactor this code base.

→ Abstract factory.

Flutter (Framework class)

setTheme()
setRefreshRate()
createUIFactory()

<< UI Factory >>

createButton() → Button
createMenu() → Menu

Android UI Factory

→ create Button → AndroidButton
→ create Menu → AndroidMenu

iOS UI Factory

→
→